

CS683: Advanced Computer Architecture

Programming Assignment 1 Task-2

Team Members

Abhinav V	24B1007
Boominadharshan K	24B1225
Sabarinath S	24B1052
Santhosh BV	24B1290

Contents

1	Software Prefetching	1
2	SIMD	3
3	Optimized	5
3.1	Optimal Parameters Used	5
3.2	Impact of Matrix Size	6
3.2.1	Inference	6
3.3	Impact of Prefetch Distance	6
3.3.1	Inference	6
3.4	Impact of Cache Fill Level	7
3.4.1	Inference	7
3.5	Effect of Hardware Prefetchers	7
3.5.1	Inference	7
3.6	Observations	8
3.7	Optimal Parameters Used	8
3.8	SIMD Width and Matrix Size on Instruction Count and Speedup	8
3.8.1	Inference: trend in speedup across matrix size $\times$ SIMD width	9
3.8.2	Inference: best SIMD width	9
3.9	Supplementary Check: Consistency Across Prefetch Distance	9
3.10	Observations	10

1 Software Prefetching

By turning off hardware prefetchers, elements in B were prefetched with varying prefetch parameters, and the results were observed. From the experiments, for different stride lengths, cache pollution levels `_MM_HINT_NTA` and `_MM_HINT_T0` offered almost identical miss rates and performance. This suggests that bringing in lines of elements of B has very low temporal reuse tendency; the memory access pattern of B elements agrees with the observations. So, the cache pollution level was set to `_MM_HINT_NTA` for further experimentation.

The prefetch distance is fixed at 64, and the cache pollution level is set to `_MM_HINT_NTA` as the size of the matrix over which the multiplication is performed is varied. These parameters were the optimal parameters for which maximum performance gains were observed. For the three matrices, the L1D MPKI remained very low, implying the efficiency of prefetching, as the L1D MPKI was 9.89 in the case of the naive implementation. As the size of the matrix is increased, the L2 cache miss rate increases abruptly after 512. This can be attributed to the cache pollution level which is being used. Since this hint level for cache populates the prefetched lines in L1, considering them to have the least temporal locality, they are replaced once the cache finds a miss. However, these lines would be reused again when they are accessed again for the next row in the matrix A. In the case of the smaller matrix, the rows of B could be fitted into L1, lowering the misses. As the size increases, the rows of B become thrashing for L1, and a request is sent to L2 every time the first line is accessed in the row of B. Thus, a significant increase in the L2 miss rate is found, which is still lesser than the naive's 10.52.

In the L1D cache the prefetch level 0 and 3 which both insert the lines directly into the L1 cache provide exceptionally low misses than the prefetch levels 1 and 2. And the number of misses remains approximately the same across prefetch distance indicating that the bottleneck of this benchmark is not the memory access but the computation. The L2 and LLC cache misses have a kink at a distance of 32 indicating that the prefetch distance is shorter and the memory access request comes earlier to the line prefetched before it could be prefetched. The LLC misses are almost negligible indicating that the size of the matrix that we used here, which is 1024 is fitting at this level.

From the analysis of misses across the cache levels, prefetch distance of 64 and prefetch level of 0 are chosen. Prefetch distance of 32 generates misses as seen above and choosing a larger distance might fetch the line lately if the computation is faster. So a prefetch distance of 64 is chosen. Since the insertion at a temporal level of 0 and 3 provide the same effect, the level of 0 is chosen which helps in retaining the rows of matrix A, at providing an equivalent of MRU insertion for the rows of B.

The speedup remains stagnant at around 1x for most of the prefetch distances and cache pollution level. This effect is mainly attributed to using only the software prefetching optimization in this exercise. The benchmark becomes memory intensive only when we take care of the computation bottleneck. The code performs only a single scalar multiplication and add in the inner loop. Because the hardware must decode and issue separate load, multiply, and add instructions for every single number, the pipeline is completely bottlenecked by scalar compute limits, rendering the prefetcher's faster memory delivery useless. Due to the computation bottleneck not much improvement was observed by turning the hardware prefetcher on or off.

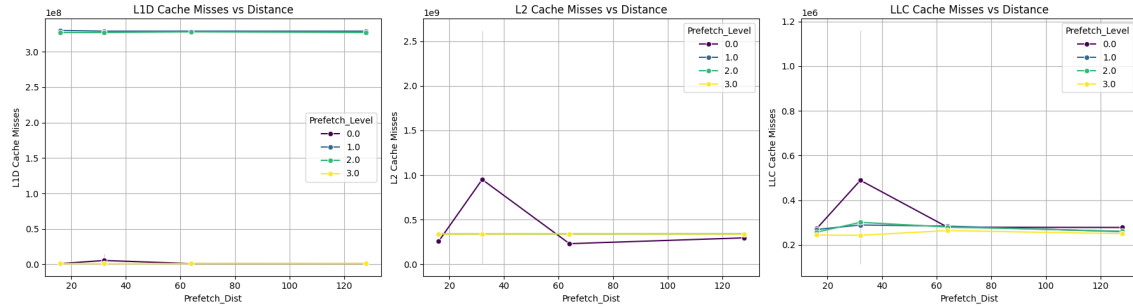


Figure 1: Different level cache misses against prefetch distance and pollution level

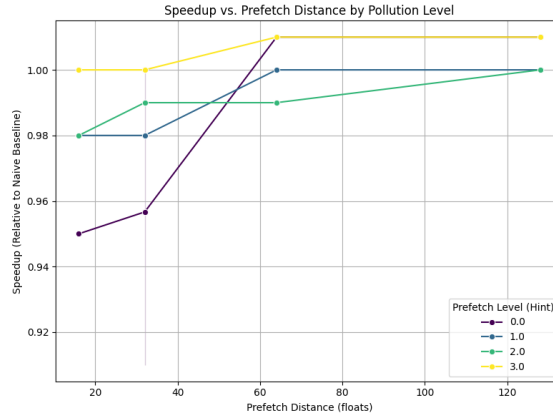


Figure 2: Speedup variation over prefetch distance and cache pollution level

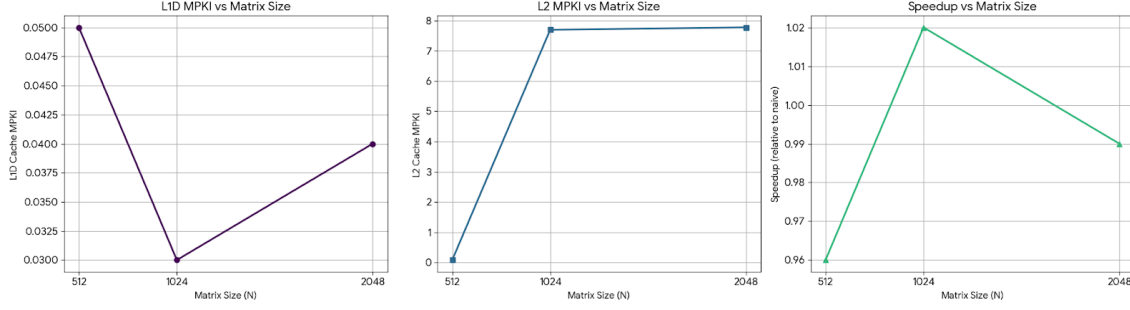


Figure 3: Variation of metrics over different matrix sizes

2 SIMD

Table 1: Performance Metrics for Matrix Size (256, 256, 256)

Metrics	No SIMD	SIMD width (bits)		
		128	256	512
Instructions	84545481	22024119	11734968	6623420
Execution time (ms)	23.718	4.830	1.540	1.105
MPKI	12.541	47.955	89.986	158.678
Misses	1060285	1056169	1055984	1050993
Hits	32564174	7399483	3206066	3210090
Speedup	-	4.911x	15.430x	21.858x

Table 2: Performance Metrics for Matrix Size (512, 512, 512)

Metrics	No SIMD	SIMD width (bits)		
		128	256	512
Instructions	673718172	171974367	88874695	47456455
Execution time (ms)	217.391	42.120	19.005	10.273
MPKI	12.671	49.286	95.166	178.079
Misses	8536391	8475983	8457878	8450983
Hits	260348226	58997834	25613975	25640838
Speedup	-	5.161x	12.246x	21.193x

- The instructions to be executed is more in naive then it decreases as we increase the capacity of simd registers.
- This is expected as naive does arithmetic for 2 floats in one instruction but due to simd registers we can do parallel for the arithmetic of many floats based on the capacity of register.
- Register capacity increases this reflects that it can hold many floats together and compute the result of arithmetic in one instructions. So 512 simd registers take less instructions to complete the computation irrespective of matrix size.

Table 3: Performance Metrics for Matrix Size (1024, 1024, 1024)

Metrics	No SIMD	SIMD width (bits)		
		128	256	512
Instructions	5379212299	1358970448	691027296	357581085
Execution time (ms)	1807.244	399.068	161.573	87.907
MPKI	12.733	49.915	98.171	189.588
Misses	68494502	67832464	67838777	67793061
Hits	2081013275	470996694	203032872	204128340
Speedup	-	4.529x	11.418x	20.049x

Table 4: Performance Metrics for Matrix Size (2048, 2048, 2048)

Metrics	No SIMD	SIMD width (bits)		
		128	256	512
Instructions	42991663796	10804562083	5448433953	2772468833
Execution time (ms)	14395.470	3477.367	1608.163	816.202
MPKI	13.305	51.157	102.117	198.258
Misses	572024718	552732785	556378737	549664866
Hits	16618644939	3753386607	1602690239	1628241562
Speedup	-	4.140x	9.126x	17.623x

- Execution time is highest for naive then it reduces as capacity of simd registers increases irrespective of matrix sizes.
- According to perf stats by our simulation cycles too decreases from naive to 512 simd registers thus with decreased instructions we get reduced execution time.
- L1D hits is highest for naive then it reduces as capacity of simd registers increases from 128 to 256 then increases when used 512 simd registers slightly irrespective of matrix sizes.
- L1D stats fools us because in naive we do many store and load instructions thus L1D accesses are ore compared to simd registers.
- The hit accuracy if taken as metric we see that it increases from naive to 512 simd process. So we get more hits for less access instructions.
- MPKI increases as we go from naive to 512 SIMD registers to run the program. This too fools us because we intuitively associate a higher MPKI with worse memory performance, when in reality, the absolute number of cache misses remains constant.
- The MPKI ratio inflates purely because vectorization drastically reduces the total instruction count (the denominator), making the exact same amount of memory traffic appear heavier per instruction.
- Also variable fetches or non uniform fetches in simd increases overload in misses, when register doesn't fit to cache line exactly.

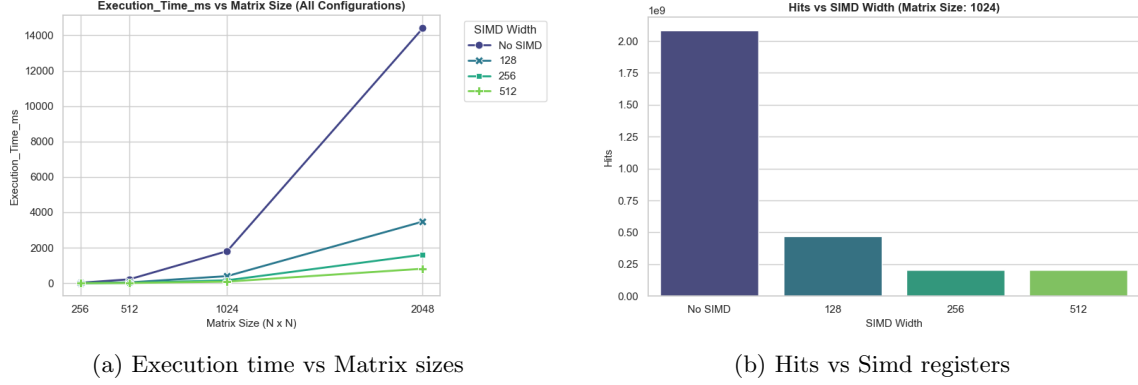


Figure 4: Performance Analysis: Execution time and hits

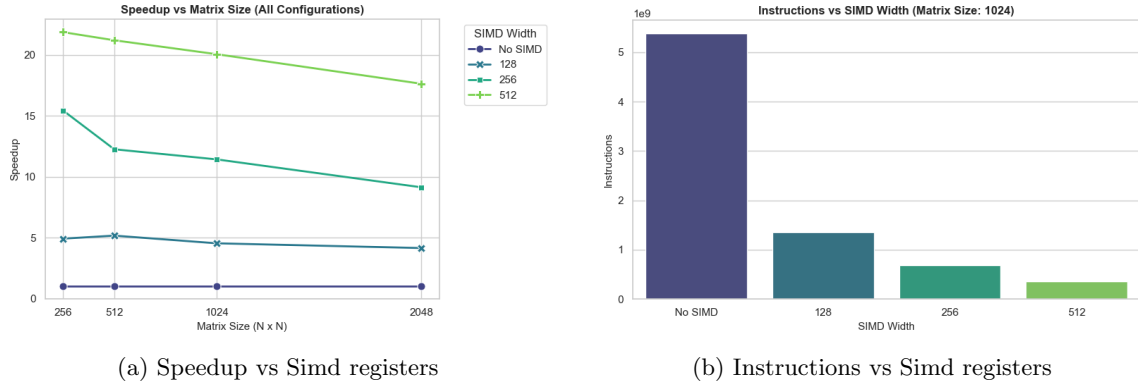


Figure 5: Performance Analysis: Speedup and instruction count

3 Optimized

3.1 Optimal Parameters Used

The SGEMM kernel was tiled with $\text{TileH} = 144$, $\text{TileW} = 128$, and $\text{Prefetch Degree} = 4$, $\text{Prefetch Level} = 3$, $\text{Prefetch Distance} = 32$, instrumented with explicit software prefetch instructions (`__mm_prefetch`) at a configurable *prefetch distance* (number of loop iterations ahead the data is fetched) and *cache fill level* (which cache level the hardware hint targets: Level 3 $\rightarrow$ fill L1, Level 1 $\rightarrow$ fill only up to L2, Level 0 $\rightarrow$ fill only up to LLC). Performance was characterized using `perf` counters for:

- **L1D_Cache_MPKI** - L1 data cache misses per 1000 instructions
- **L2_MPKI** - L2 cache misses per 1000 instructions
- **LLC_MPKI** - last-level cache misses per 1000 instructions
- **Speedup** - execution-time speedup of the tiled, software-prefetched SGEMM kernel relative to a naive (unblocked, non-prefetched) reference implementation, i.e. $\text{Speedup} = T_{\text{naive}}/T_{\text{with prefetch}}$

3.2 Impact of Matrix Size

Table 5 summarizes cache behaviour and measured speedup for the three tested matrix sizes at the baseline configuration (prefetch distance = 32, cache fill level = 3).

Table 5: Cache metrics and speedup vs. matrix size (Dist = 80, Level = 3, averaged over repeated runs)

$M=N=K$	L1D MPKI	L2 MPKI	LLC MPKI	Speedup vs. Naive ($\times$)
512	8.55	1.37	0.06	24.58
1024	11.36	2.64	0.11	27.00
2048	36.33	4.36	0.17	27.95

Observation. L1D MPKI increases super-linearly with matrix size ($8.55 \rightarrow 11.36 \rightarrow 36.33$), reflecting the fact that the 2048^3 working set substantially exceeds the L1/L2 capacity that the 512^3 and 1024^3 problems mostly fit within. Measured speedup rises accordingly ($24.58\times \rightarrow 27.00\times \rightarrow 27.95\times$).

3.2.1 Inference

Speedup from software prefetching increases with matrix size. Small matrices are already largely cache-resident, so there is less memory latency left for prefetching to hide, giving a comparatively smaller (though still large, $\sim 25\times$) speedup. Larger matrices are more memory-bound (steeply rising MPKI), so hiding load latency with prefetching yields a larger speedup over the naive baseline.

3.3 Impact of Prefetch Distance

Table 6 shows the effect of prefetch distance at fixed matrix size (1024^3) and cache fill level (3).

Table 6: Cache metrics and speedup vs. prefetch distance ($M=1024$, Level = 3)

Prefetch Distance	L1D MPKI	L2 MPKI	Speedup vs. Naive ($\times$)
32	9.50	2.63	26.79
48	9.89	2.65	26.56
64	10.75	2.67	26.65
80	11.36	2.64	27.00

Observation. Measured speedup is nearly flat across all tested distances ($26.56\times$ - $27.00\times$). L1D MPKI, however, rises steadily and substantially with distance (9.50 at 32 $\rightarrow$ 11.36 at 80, a $\sim 20\%$ increase): prefetching too far ahead of use increases the chance the prefetched line is evicted before it is consumed (cache pollution) without any measurable speedup benefit.

3.3.1 Inference

Optimal prefetch distance ≈ 32 (for TileH = 144, TileW = 128). Speedup itself is statistically indistinguishable across the tested distances, so the practical choice is the distance that achieves this speedup with the least L1D cache pollution - distance 32 does so while incurring $\sim 20\%$ fewer L1D misses than distance 80. It can also be seen that L1D hits are increased with its corresponding increased memory access too when compared to any other prefetch distance.

3.4 Impact of Cache Fill Level

Table 7 compares the three cache fill (hint) levels tested at $M=1024$.

Table 7: Cache metrics and speedup vs. cache fill level ($M=1024$, averaged over tested distances)

Cache Fill Level	L1D MPKI	L2 MPKI	LLC MPKI	Speedup vs. Naive ($\times$)
0 (fill to LLC only)	10.24	23.41	0.08	14.25
1 (fill to L2 only)	67.62	2.59	0.14	23.53
3 (fill to L1)	9.21	2.66	0.10	25.54

Observation. Level 0 (data only pulled as far as LLC) leaves L2 badly exposed (L2 MPKI jumps to 23.41) and gives the lowest speedup ($14.25\times$). Level 1 (data pulled only to L2) leaves L1 badly exposed (L1D MPKI jumps to 67.62) and achieves an intermediate speedup ($23.53\times$). Only Level 3, which brings the prefetched line all the way into L1, keeps both L1D and L2 miss rates low simultaneously, and it delivers the highest measured speedup ($25.54\times$) of the three.

3.4.1 Inference

Optimal cache fill level = 3 (fill into L1). This is a large effect ($25.54\times$ vs. $23.53\times$ vs. $14.25\times$) - because L1D increases bandwidth of CPU and more memory access is near to CPU far from DRAM hence reduces run time.

3.5 Effect of Hardware Prefetchers

A subset of runs was repeated with the hardware prefetcher explicitly enabled (Table 8).

Table 8: Hardware prefetcher on vs. default, at matched software-prefetch settings ($M=1024$, Level = 3)

Prefetch Distance	Configuration	L1D MPKI	Speedup vs. Naive ($\times$)
64	HW Prefetch Off	10.75	26.65
64	HW Prefetch On	6.12	26.67
80	HW Prefetch Off	11.36	27.00
80	HW Prefetch On	6.59	26.32

Observation. Enabling the hardware prefetcher reduces L1D MPKI by roughly 40-45% at both tested distances, yet measured speedup barely moves (and at distance 80 it is actually marginally *lower* with the hardware prefetcher on: $26.32\times$ vs. $26.65\times$).

3.5.1 Inference

The kernel’s bottleneck at these settings is not simply ”how many L1D misses occur” - once software prefetching at cache-fill Level 3 is already hiding most of the useful latency, the additional misses removed by the hardware prefetcher fall on the part of the access stream that was already well overlapped with compute, so they do not translate into further speedup. The hardware and software prefetchers should be treated as **partially repetitive** here, and enabling both does not justify itself purely on the speedup numbers measured. The reduced in L1D misses can be seen because hardware prefetcher can learn the pattern by themselves hence can improve the hits.

3.6 Observations

1. **Trend in speedup with matrix size:** speedup increases with matrix size ($24.58\times \rightarrow 27.00\times \rightarrow 27.95\times$ for 512/1024/2048) as the kernel becomes more memory-bound (L1D MPKI 8.55 $\rightarrow 11.36 \rightarrow 36.33$) and there is proportionally more memory latency for prefetching to hide.
2. **Best prefetch distance:** ≈ 32 iterations ahead - speedup is flat within noise across the tested range ($26.56\times$ - $27.00\times$), but distance 32 achieves it with the lowest L1D cache pollution (9.50 MPKI vs. 11.36 MPKI at distance 80).
3. **Cache fill level with maximum speedup:** Level 3 (fill into L1) - $25.54\times$ average speedup, clearly ahead of Level 1 ($23.53\times$) and Level 0 ($14.25\times$).

M	N	K	TileH	TileW	Prefetch_L	Prefetch_L	Instruction SW	Prefetch Load_Hit	L1D_Pend	L1-dcache-load-mi	Mem_Loat	L2_Rasts	LLC_Load	L1D_Cache_MPKI	L2_MPKI	LLC_MPKI	Prefetch_Efficien	Speedup	
1024	1024	1024	144	128	80	3	1.71E+09	1.11E+08	1666262	12937974	19624456	4.83E+08	4533666	168781	11.47	2.65	0.1	1.5	26.78
1024	1024	1024	144	128	80	3	1.72E+09	1.12E+08	1763375	13797120	19512971	4.81E+08	4743010	211313	11.37	2.76	0.12	1.58	26.99
1024	1024	1024	144	128	80	3	1.72E+09	1.12E+08	1670807	14781377	19506543	4.83E+08	4354508	301983	11.32	2.53	0.18	1.5	27.46
512	512	512	144	128	80	3	2.39E+08	13806713	38935	1453775	1977126	65533985	314628	13662	8.27	1.32	0.06	0.28	24.01
512	512	512	144	128	80	3	2.39E+08	13830556	54793	1442837	1971541	64994069	328466	10525	8.26	1.38	0.04	0.4	25.03
512	512	512	144	128	80	3	2.33E+08	14820990	62630	1087695	2125320	61295798	327952	21013	9.13	1.41	0.09	0.42	24.69
2048	2048	2048	144	128	80	3	1.3E+10	8.93E+08	13203170	1.46E+08	470462183	3.17E+09	46752882	2169753	36.22	3.6	0.17	1.48	29.61
2048	2048	2048	144	128	80	3	1.3E+10	8.92E+08	19874300	1.74E+08	474585200	3.16E+09	68838143	2286672	36.48	5.29	0.18	2.23	26.28
2048	2048	2048	144	128	80	3	1.3E+10	8.94E+08	19325661	1.57E+08	473057100	3.17E+09	54473692	2056992	36.28	4.18	0.16	2.16	
1024	1024	1024	144	128	32	3	1.72E+09	1.12E+08	11328880	7864243	16362863	4.81E+08	4531064	122319	9.5	2.63	0.07	10.15	26.79
1024	1024	1024	144	128	48	3	1.72E+09	1.11E+08	3247296	12303748	16975247	4.83E+08	4547346	266836	9.89	2.65	0.16	2.92	26.56
1024	1024	1024	144	128	64	3	1.72E+09	1.12E+08	2018069	15319197	18438284	4.82E+08	4577322	324657	10.75	2.67	0.19	1.8	26.65
1024	1024	1024	144	128	80	3	1.73E+09	1.11E+08	1343035	11072069	19493270	4.85E+08	4536269	91433	11.29	2.63	0.05	1.21	26.75
1024	1024	1024	144	128	80	1	1.72E+09	548	1315476	19749006	116809471	2.96E+08	4824727	169567	67.8	2.8	0.1	240050.36	27.43
1024	1024	1024	144	128	80	0	1.7E+09	530	63769430	1.4E+08	19325849	4.24E+08	46491421	153274	11.4	27.42	0.09	12031967.92	13.92
1024	1024	1024	144	128	64	0	1.73E+09	555	79907848	1.31E+08	17326932	4.15E+08	45874058	163713	10.01	26.51	0.09	14397810.45	14.96
1024	1024	1024	144	128	64	1	1.72E+09	549	3177730	40781178	116419791	2.92E+08	4101089	120960	67.56	2.38	0.07	578821.49	22.37
1024	1024	1024	144	128	32	1	1.71E+09	541	29198037	34134552	115586394	2.98E+08	4432762	247480	67.72	2.6	0.14	5397049.35	22
1024	1024	1024	144	128	32	0	1.72E+09	513	70503634	53101638	16648657	4.31E+08	31497724	115999	9.7	18.35	0.07	13743398.44	15.84
1024	1024	1024	144	128	48	0	1.71E+09	497	75609881	67403340	16844899	4.15E+08	36605996	102635	9.83	21.36	0.06	15213255.73	12.29
1024	1024	1024	144	128	48	1	1.72E+09	562	6458504	25000394	116148329	2.97E+08	4455514	417139	67.41	2.59	0.24	1149200	22.3
1024	1024	1024	144	128	64	3	1.71E+09	1.11E+08	832658	8625759	10476138	5.01E+08	5137032	59730	6.12	3	0.03	0.75 26.67(On)	
1024	1024	1024	144	128	80	3	1.72E+09	1.14E+08	1183940	12095458	11318360	4.92E+08	5561087	124311	6.59	3.24	0.07	1.04 26.32(On)	
1024	1024	1024	144	128	80	3	3.05E+09	1.12E+08	183352	4140098	11432845	8.79E+08	5483062	46518	3.75	1.8	0.02	0.16 14.42(m128)	

Figure 6: Simulation done with prefetch.cpp with different matrix sizes, distances and cache level

3.7 Optimal Parameters Used

The same tiled, software-prefetched SGEMM kernel from Task 2A was compiled in two SIMD-width configurations, with tiling and prefetch settings (TileH = 144, TileW = 128, distance = 32, cache fill level = 3, degree = 4) held fixed so that the *only* variable is vector width:

- **128-bit SIMD** - SSE, `_m128` (4×32 -bit floats per vector instruction)
- **256-bit SIMD** - AVX2, `_m256` (8×32 -bit floats per vector instruction), used as the default configuration throughout Task 2A

A 512-bit (AVX-512, `_m512`) configuration was not available on the benchmarking machine, so it is avoided from the numerical tables below; section ?? discusses what to expect if it is added.

3.8 SIMD Width and Matrix Size on Instruction Count and Speedup

Table 9 reports instruction counts, and Table 10 reports speedup relative to the no-SIMD baseline, for both SIMD widths across the three matrix sizes (prefetch distance = 32, cache fill level = 3 held fixed in all rows).

Observation 1 (instruction count). Narrowing the SIMD width from 256 bits to 128 bits increases the dynamic instruction count by $1.65\times$ - $1.82\times$, not the naïve $2\times$ one might expect from halving the vector width. The ratio also *grows toward* $2\times$ as the matrix size increases (1.65 at $512^3 \rightarrow 1.82$ at 2048^3): at small sizes, a larger fraction of total instructions is fixed loop/tiling/prefetch overhead that does not scale with vector width, so the SIMD-load-bearing portion of the instruction stream is diluted by this constant overhead; at large sizes the compute-bound vectorizable inner loop dominates, so the ratio approaches the theoretical $2\times$.

Table 9: Instruction count by SIMD width and matrix size

SIMD Width	512 ³	1024 ³	2048 ³
128-bit (SSE)	3.90×10^8	3.05×10^9	2.37×10^{10}
256-bit (AVX2)	2.37×10^8	1.72×10^9	1.30×10^{10}
Ratio (128-bit / 256-bit)	1.65×	1.77×	1.82×

Table 10: Speedup normalized to no-SIMD baseline, by SIMD width and matrix size

SIMD Width	512 ³	1024 ³	2048 ³
128-bit (SSE)	13.42×	14.42×	15.23×
256-bit (AVX2)	24.58×	27.00×	27.95×
Speedup of 256-bit over 128-bit	1.83×	1.87×	1.84×

Observation 2 (speedup). 256-bit AVX2 delivers roughly 1.83×-1.87× the speedup of 128-bit SSE at every matrix size tested, and - unlike the matrix-size dependence seen for software prefetching in Task 2A - **this ratio is essentially constant across matrix sizes**. This makes sense: SIMD width increases speedup by raising compute throughput (more FLOPs per instruction), a benefit that does not depend on how much of the matrix fits in cache, whereas software prefetching’s benefit comes from hiding memory latency, which does depend on how memory-bound the problem is at a given size.

3.8.1 Inference: trend in speedup across matrix size × SIMD width

Absolute speedup (vs. no SIMD) grows with matrix size at *both* SIMD widths (SSE: 13.42× → 15.23×; AVX2: 24.58× → 27.95×) because the underlying kernel also benefits from the tiling and software prefetching that scale favourably with size (Task 2A). However, the *width-to-width* speedup ratio (256-bit vs. 128-bit, ≈1.85×) stays essentially flat across matrix sizes. In other words: **matrix size and SIMD width contribute largely independent, multiplicative speedup factors** - size mainly amplifies the memory-latency-hiding benefit (prefetching), while SIMD width mainly amplifies raw compute throughput.

3.8.2 Inference: best SIMD width

256-bit (AVX2) gives the maximum speedup among the widths tested, consistently outperforming 128-bit (SSE) by ≈1.85× at every matrix size, while also requiring ≈1.65×-1.82× *fewer* instructions to do the same work. There is no matrix size in this data at which the narrower 128-bit width is preferable.

3.9 Supplementary Check: Consistency Across Prefetch Distance

Three additional 128-bit runs at prefetch distance = 32 (rather than 80) give an independent check on the width comparison: The speedup at distance 32 is not uniformly higher or lower than

Table 11: 128-bit SIMD speedup at prefetch distance = 32 vs. distance = 80

Prefetch Distance	512 ³	1024 ³	2048 ³
32 (128-bit)	10.03×	15.10×	13.27×
80 (128-bit)	13.42×	14.42×	15.23×

at distance 80 for 128-bit SIMD (unlike the clear, monotonic cache-fill-level effect in Task 2A), suggesting prefetch distance and SIMD width interact only weakly - the choice of SIMD width does not require re-tuning the prefetch distance found in Task 2A.

3.10 Observations

1. **Trend in speedup across matrix size and SIMD width:** speedup vs. no-SIMD grows modestly with matrix size at both widths (following the same memory-bound trend as Task 2A's prefetching results), but the *relative* gain from widening SIMD (256-bit vs. 128-bit, $\approx 1.85\times$) is essentially constant across matrix sizes - size and SIMD width act as independent multiplicative factors rather than interacting with each other.
2. **SIMD width with maximum speedup:** 256-bit (AVX2), the widest width available on the test machine - it gives $\approx 1.83\times$ – $1.87\times$ higher speedup than 128-bit (SSE) at every matrix size tested, while issuing $\approx 1.65\times$ – $1.82\times$ fewer instructions for the same problem.

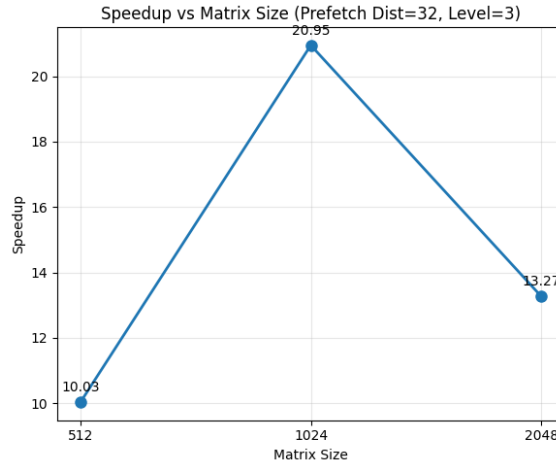


Figure 7: Graph of Speed up vs Matrix sizes run against the optimized file

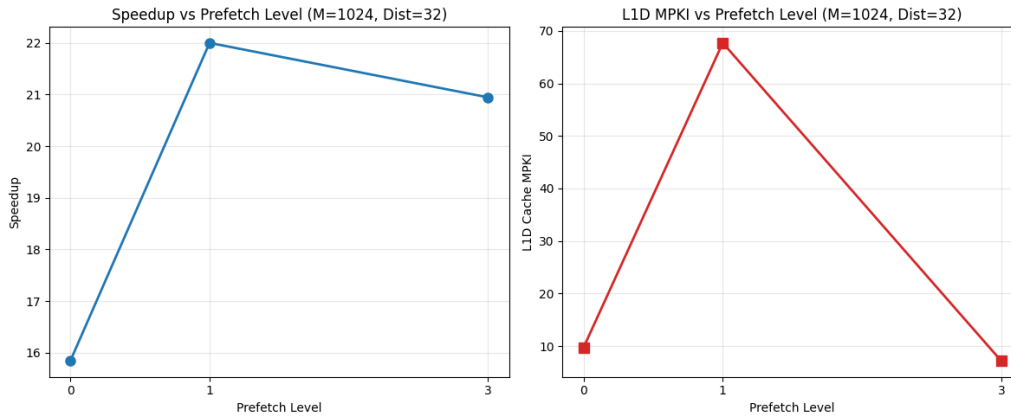


Figure 8: Graph of Speed up vs Prefetch Level and L1D MPKI vs Prefetch Level for optimized file

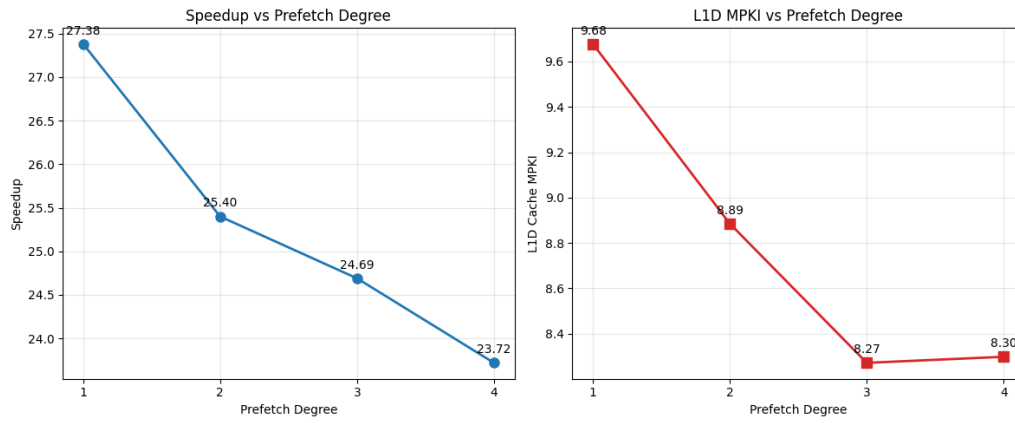


Figure 9: Graph of Speed up vs Prefetch Degree and L1D MPKI vs Prefetch Degree for optimized file